ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN
***

BÁO CÁO ĐỒ ÁN MÔN HỌC
CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

NHẬT KÝ CÔNG VIỆC NHÓM 24

GIẢNG VIÊN HƯỚNG DẪN: PHẠM THI VƯƠNG
NHÓM THỰC HIỆN: NHÓM 24
Lê Quốc Cường — MSSV: 3123410040
Huỳnh Thanh Hải — MSSV: 3123410091
Nguyễn Phú Tài — MSSV: 3121410435

Hồ Chí Minh tháng 11 năm 2026

NHẬT KÝ CÔNG VIỆC NHÓM 24 1
1. Tổng quan Dự án & Hồ sơ Repository Git 3
2. Nhật ký Tiến độ 12 Tuần & Phân công Trách nhiệm Chi tiết 4
3. Phân tích Chi tiết Lịch sử Git Commit theo Các Mốc Kiến trúc 6
3.1. Các Mốc Commit Cốt lõi trên Nhánh Main 6
3.2. Đóng góp trên Nhánh phu-tai (Frontend Client & Infrastructure) 7
3.3. Minh bạch về Tài khoản Contributor trên GitHub 7
4. Ma trận Đồng bộ Kỹ thuật: Git Commits — Code Spring Boot — Báo cáo 8
4.1. Tầng 1 — Các Yêu cầu Bắt buộc Lõi (Spring Boot Core) 8
4.2. Tầng 2 & 3 — Kỹ thuật Nâng cao & Xử lý Đồng thời 8
5. Quy trình Kiểm thử Tự động, Minh chứng Thực thi & Thí nghiệm Độc lập 9
5.1. Kết quả Bộ Kiểm thử Tự động JUnit (./mvnw clean verify) 9
5.2. Kết quả Kiểm thử E2E Smoke qua Mạng Thật (tools/e2e_check.py) 10
5.3. Thí nghiệm SQL Lab Tách biệt (Đo đạc Tầng CSDL) 10
6. Bài học Kinh nghiệm & Nhật ký Sửa lỗi (Debugging Case Studies) 11
Tình huống 1: Lỗi Mất Principal giữa các Khung truyền STOMP 11
Tình huống 2: Giữ đúng totalElements khi Trang Lịch sử Rỗng 11
Tình huống 3: Lỗi Bất đồng Hợp đồng Destination STOMP giữa Client và Backend 12
7. Đánh giá Tổng kết & Định hướng nâng cấp 12
Hướng Phát triển Nâng cấp trong Tương lai 12

Tài liệu được nhóm hệ thống hóa toàn bộ tiến trình phát triển, phân công trách nhiệm và đối chiếu vết mã nguồn của dự án NextChat Backend — hệ thống nhắn tin thời gian thực xây dựng trên nền tảng Spring Boot 4.1.1 (Java 21) và PostgreSQL 16. Tài liệu tích hợp dữ liệu thực tế từ lịch sử Git commit trên repository springboot-websocket-chat, hồ sơ báo cáo đồ án nhóm, báo cáo cá nhân và các biên bản thực thi kiểm thử nhằm cung cấp một bức tranh toàn diện, minh bạch và đạt chuẩn kiểm thử sản xuất (production-ready).

1. Tổng quan Dự án & Hồ sơ Repository Git

Dự án NextChat được phát triển trong khuôn khổ học phần *Các công nghệ lập trình hiện đại* (Mã HP: 841072) tại Khoa Công nghệ Thông tin, Trường Đại học Sài Gòn, dưới sự hướng dẫn của giảng viên Phạm Thi Vương.

1. Tên ứng dụng: NextChat Real-time Messaging Backend
2. Repository GitHub chính thức: https://github.com/QuoocsCuongwf/springboot-websocket-chat
3. Nhánh phát triển chính (Main Baseline): Snapshot niêm phong 39b4d75 (docs(report): align report with implemented features)
4. Nhánh phát triển giao diện client: Nhánh phu-tai (chứa commit 4699986 và 332d069)
5. Thành viên Nhóm 24:
    5.1. Lê Quốc Cường (MSSV: 3123410040) — Trưởng nhóm / Architect & Lead Developer (Tài khoản GitHub: QuoocsCuongwf)
    5.2. Huỳnh Thanh Hải (MSSV: 3123410091) — Core Backend & Database Engineer (Tài khoản GitHub: ThanhHai151)
    5.3. Nguyễn Phú Tài (MSSV: 3121410435) — Frontend Client & Infrastructure Developer (Tác giả commit 4699986 trên nhánh phu-tai)

2. Nhật ký Tiến độ 12 Tuần & Phân công Trách nhiệm Chi tiết

Tiến độ đồ án được thực hiện nghiêm túc qua 12 tuần theo lịch học phần. Bảng dưới đây đối chiếu chi tiết giữa kế hoạch phân công, người phụ trách chính, mốc thời gian thực tế và kết quả mã nguồn đạt được trên Git:

Tuần	Công việc & Mốc kỹ thuật	Thành viên phụ trách	Trạng thái & Vết mã nguồn (Git / Output)
Tuần 1–2 (11/09–24/09)	Khảo sát bài toán chat real-time; chốt công nghệ Java 21, Spring Boot 4.1.1, WebSocket/STOMP,	Lê Quốc Cường (Phụ trách),	Hoàn thành. Khởi tạo repository springboot-websocket-c

Tuần	Công việc & Mốc kỹ thuật	Thành viên phụ trách	Trạng thái & Vết mã nguồn (Git / Output)
	PostgreSQL 16, JWT; khởi tạo kho mã.	Huỳnh Thanh Hải, Nguyễn Phú Tài	hat; commit đầu tiên 15b9513 (Initial commit) / 6ab071a ngày 22/09/2026.
Tuần 3 (25/09–01/10)	Thiết lập cấu hình Spring Boot, tổ chức package theo chức năng (package-by-feature), cấu hình Docker Compose cho PostgreSQL 16-alpine (kèm healthcheck pg_isready), viết migration Flyway V1__messenger_schema.sql (20 bảng).	Huỳnh Thanh Hải (Database/Auth), Lê Quốc Cường (Core/Config)	Hoàn thành. Commits 67fb244 (tổ chức package), dfa4a3b (Docker Compose & .env.example), a2626e6 (hoàn thiện Auth, Room REST API & database schema). Ánh xạ 52 thuộc tính cột của 6 JPA Entity.
Tuần 4 (02/10–08/10)	Cấu hình WebSocket Message Broker (WebSocketConfig) với endpoint /ws và /ws-chat, xây dựng RoomWebSocketController và hàm PL/pgSQL phân trang tin nhắn get_room_messages.	Lê Quốc Cường (WebSocket), Huỳnh Thanh Hải (PL/pgSQL)	Hoàn thành. Commits deabd07 (cấu hình WebSocket), c3acb71 (thêm STOMP controller /room.{roomId}), 5ea2fc2 (merge main into branch feat/auth-websocket).
Tuần 5 (09/10–15/10)	Tối ưu hóa truy cập dữ liệu JPA, áp dụng @EntityGraph cho ConversationRepository chống lỗi N+1 query, xây dựng giao dịch nguyên tử @Transactional và hàm PL/pgSQL get_or_create_direct_room.	Huỳnh Thanh Hải (REST/JPA), Lê Quốc Cường (Transaction)	Hoàn thành. Rà soát hiệu năng nạp dữ liệu danh sách phòng (getUserRooms), xây dựng cơ chế bọc DTO trong phạm vi giao dịch với open-in-view: false.

Tuần	Công việc & Mốc kỹ thuật	Thành viên phụ trách	Trạng thái & Vết mã nguồn (Git / Output)
Tuần 6 (16/10–22/10)	Hoàn thiện luồng Refresh Token Rotation băm SHA-256 (RefreshTokenService), cài đặt bộ lọc JwtStompChannelInterceptor bảo mật WebSocket, bổ sung bộ kiểm thử JUnit.	Lê Quốc Cường (Security/STOMP), Huỳnh Thanh Hải (Auth)	Hoàn thành. Commits d20e734 (hoàn thiện JWT STOMP security), 3eaa91a (test suite cho chat STOMP), 39b4d75 (Docs align report - niêm phong nhánh main ngày 20/10).
Tuần 7 (23/10–29/10)	Rà soát chuyên sâu báo cáo, phát hiện và sửa lỗi mất Principal trong STOMP interceptor (MessageHeaderAccessor.getAccessor), mở rộng test lên 28 JUnit PASS và 15 E2E smoke PASS, thực hiện thí nghiệm SQL Lab.	Lê Quốc Cường (Lead/QA), Huỳnh Thanh Hải (Testing)	Hoàn thành đợt rà soát. Bổ sung các bản sửa cục bộ giữ Principal, test rollback Spring (SpringMechanismIntegrationTest), unit test Mockito (RoomServiceUnitTest) và thí nghiệm SQL tools/sql_lab.py.
Tuần 8 (30/10–05/11)	Phát triển ứng dụng Client Web thử nghiệm (HTML/JS đơn trang), cấu hình tài nguyên tĩnh, đồng nhất endpoint STOMP destination (/app/room/{roomId} và /topic/room/{roomId}).	Nguyễn Phú Tài (Client UI), Lê Quốc Cường (STOMP Sync)	Hoàn thành trên nhánh phu-tai. Nguyễn Phú Tài commit 4699986 (Client web, static config, Compose tweak); Lê Quốc Cường commit 332d069 (đồng bộ endpoint STOMP).
Tuần 9 (Từ 06/11)	Đóng gói hồ sơ báo cáo đồ án nhóm, báo cáo cá nhân, nhật ký công việc, biên bản làm việc nhóm và mã nguồn nộp giảng viên.	Cả nhóm (Lê Quốc Cường tổng hợp)	Hoàn thành. Xuất bản hồ sơ chính thức Thứ 6_Nhóm 24_Final.

3. Phân tích Chi tiết Lịch sử Git Commit theo Các Mốc Kiến trúc

Lịch sử Git commit của kho lưu trữ springboot-websocket-chat phản ánh chính xác quá trình tiến hóa của mã nguồn từ nền tảng đến các tính năng nâng cao.

3.1. Các Mốc Commit Cốt lõi trên Nhánh Main

- Commit 15b9513 / 6ab071a (Initial commit): Khởi tạo repository, thiết lập file LICENSE (Apache-2.0) và .gitignore.
- Commit 67fb244 (refactor: organize package structure by feature): Tái cấu trúc thư mục mã nguồn Java theo mô hình *package-by-feature + common*. Tách biệt thành các gói com.quoocscuongwf.nextchat.auth, user, room và common.
- Commit dfa4a3b (chore: add docker-compose and environment configuration): Xây dựng file docker-compose.yml khởi chạy container PostgreSQL 16-alpine với cấu hình volume lưu trữ bền vững và healthcheck pg_isready. Tạo file mẫu .env.example phục vụ nạp biến môi trường ngoài Git.
- Commit a2626e6 (feat(api): complete auth, rooms, and database task): Commit diện rộng của Huỳnh Thanh Hải và Lê Quốc Cường: hoàn thiện các mô-đun REST API xác thực (/api/v1/auth), quản lý hồ sơ người dùng (/api/v1/users), phòng chat (/api/v1/rooms), nạp file migration Flyway V1__messenger_schema.sql (20 bảng) và file cấu hình database.yml.
- Commit deabd07 & c3acb71 (feat(websocket): configure STOMP message broker and room controller): Đăng ký WebSocket broker với endpoint /ws và /ws-chat, xây dựng RoomWebSocketController tiếp nhận tin nhắn qua @MessageMapping("/room.{roomId}") và phát broadcast qua @SendTo("/topic/room.{roomId}").
- Commit 5ea2fc2 (Merge main into branch feat/auth-websocket): Hợp nhất nhánh tính năng WebSocket và xác thực JWT vào nhánh phát triển chính.
- Commit d20e734 (feat(security): complete JWT STOMP channel interceptor): Cài đặt JwtStompChannelInterceptor trích xuất và xác thực token JWT tại khung truyền CONNECT, đồng thời kiểm tra quyền thành viên phòng chat khi client gửi lệnh SUBSCRIBE.
- Commit 3eaa91a (test(chat): cover JWT STOMP and realtime messages): Bổ sung bộ kiểm thử đơn vị cho JwtStompChannelInterceptorTest và bài test tích hợp luồng lưu trữ tin nhắn real-time.
- Commit 39b4d75 (docs(report): align report with implemented features): Commit niêm phong mã nguồn nhánh main trước đợt rà soát cuối, đồng bộ tài liệu báo cáo khớp 100% với các tính năng thực tế.

3.2. Đóng góp trên Nhánh phu-tai (Frontend Client & Infrastructure)

- Commit 4699986 (Nguyễn Phú Tài, ngày 30/10/2026):
    - Xây dựng ứng dụng Client Web thử nghiệm một trang (index.html và app.js tại src/main/resources/static) kết nối đủ 9 REST API endpoints và SockJS/STOMP.
    - Cấu hình mở đường dẫn công khai cho tài nguyên tĩnh trong WebSecurityConfig.
    - Tinh chỉnh tham số mặc định CSDL trong docker-compose.yml.
- Commit 332d069 (Lê Quốc Cường, ngày 30/10/2026):
- Đồng bộ hóa cú pháp destination STOMP giữa Client và Backend sang dạng đường dẫn chuẩn /app/room/{roomId} và /topic/room/{roomId}, tích hợp SimpMessagingTemplate trong controller để phát tin linh hoạt.

3.3. Minh bạch về Tài khoản Contributor trên GitHub

Trên giao diện GitHub Insights/Contributors của repository springboot-websocket-chat, hệ thống ghi nhận 4 tài khoản chính thức :

- QuoocsCuongwf (Lê Quốc Cường)
- ThanhHai151 (Huỳnh Thanh Hải)
- Copilot (Công cụ AI hỗ trợ)
- NguyenPhuTai254 (Nguyễn Phú Tài)

4. Ma trận Đồng bộ Kỹ thuật: Git Commits — Code Spring Boot — Báo cáo

Mã nguồn trong repository và nội dung báo cáo tuân thủ nghiêm ngặt các tiêu chuẩn kỹ thuật backend được quy định trong tài liệu hướng dẫn môn học.

4.1. Tầng 1 — Các Yêu cầu Bắt buộc Lõi (Spring Boot Core)

- Auto-Configuration & Starters : Khai báo các starter spring-boot-starter-web, data-jpa, security, websocket, flyway. Bài test SpringMechanismIntegrationTest.realBeansAreSingletonsAndAutoConfigurationMatches xác nhận bean DataSource đạt singleton và log ConditionEvaluationReport khớp đúng HikariDataSource.
- IoC Container & Constructor Injection : Toàn bộ các dependency trong Controller và Service được tiêm qua hàm khởi tạo (Constructor Injection). Ứng dụng chỉ định nghĩa 5 Bean cấu hình bảo mật/dữ liệu (WebSecurityConfig, PasswordConfig, SeedDataConfig, WebSocketConfig), hoàn toàn không tạo DataSource hay EntityManagerFactory thủ công.

- Cấu hình ngoài & Profile : Quản lý thuộc tính qua application.yaml, database.yml và .env. JwtService có cơ chế tự ngắt ứng dụng ngay khi khởi động nếu phát hiện khóa JWT secret mặc định bị sử dụng ngoài môi trường dev và test.
- Spring Security & Stateless JWT : Bộ lọc JwtAuthenticationFilter tra cứu trực tiếp CSDL qua DatabaseUserDetailsService (chỉ chấp nhận isActive && !isDeleted). Nếu tài khoản bị khóa, mọi access token còn hạn lập tức bị từ chối ở request kế tiếp.
- Spring Data JPA & Chống lỗi N+1 Query : Tắt open-in-view: false trong database.yml. Sử dụng @EntityGraph trong ConversationRepository.findRoomsByUserId để nạp danh sách phòng kèm thành viên và tin nhắn cuối trong đúng 1–2 câu lệnh SQL.
- Quản lý CSDL bằng Flyway & ddl-auto: validate : Flyway thực thi hai file migration (V1__messenger_schema.sql và V2__lock_direct_room_creation.sql). Hibernate chạy chế độ validate đối chiếu khớp 52/52 thuộc tính cột của 6 JPA Entity chính (User, Role, RefreshToken, Conversation, ConversationParticipant, Message).

4.2. Tầng 2 & 3 — Kỹ thuật Nâng cao & Xử lý Đồng thời

- Triệt tiêu Race Condition bằng Khóa Advisory CSDL:
  - *Bài toán*: Hai người dùng A và B bấm nhắn tin cho nhau tại cùng một mili-giây, dẫn đến nguy cơ tạo 2 phòng chat 1-1 trùng lặp.
- *Giải pháp*: Xây dựng hàm PL/pgSQL get_or_create_direct_room tích hợp khóa tư vấn mức giao dịch pg_advisory_xact_lock trên mã băm chuẩn hóa thứ tự ID:
-

```SQL
PERFORM pg_advisory_xact_lock(
   hashtextextended(
      LEAST(p_user1_id, p_user2_id)::text || ':' || GREATEST(p_user1_id,
p_user2_id)::text,
      0
   )
);
```

- Refresh Token Rotation băm SHA-256:
  - RefreshTokenService.rotateRefreshToken thực thi trong giao dịch @Transactional. Token thô 32 bytes (Base64URL) gửi cho client, nhưng chỉ lưu bản băm SHA-256 (43 ký tự Base64URL) trong CSDL.

    - Khi client gọi /api/v1/auth/refresh, token cũ bị xóa ngay lập tức và token mới được phát hành. Nếu token cũ bị kẻ xấu dùng lại, hệ thống từ chối với lỗi 401 Unauthorized do bản băm cũ không còn tồn tại.
- WebSocket / STOMP Interceptor Security:
    - JwtStompChannelInterceptor xác thực JWT tại frame CONNECT.
    - Tại frame SUBSCRIBE tới /topic/room.{roomId}, interceptor tra cứu CSDL xác nhận người dùng đang là thành viên active (left_at IS NULL) của phòng.
    - Tại frame SEND, interceptor ép buộc destination phải khớp mẫu /app/room.{roomId}, ngăn chặn hoàn toàn việc client gửi tin trực tiếp vào broker bypass qua tầng nghiệp vụ.

5. Quy trình Kiểm thử Tự động, Minh chứng Thực thi & Thí nghiệm Độc lập

Hệ thống được bảo chứng bằng 3 cấp độ kiểm thử minh bạch:

5.1. Kết quả Bộ Kiểm thử Tự động JUnit (./mvnw clean verify)

Lần thực thi ngày 28/10/2026 ghi nhận 28/28 bài test PASS (0 Failures, 0 Errors, 0 Skipped):

- AuthAndRoomIntegrationTest (18 bài test tích hợp) : Vận hành trên CSDL postgres:16-alpine thật được khởi tạo tự động qua Testcontainers . Bao phủ toàn bộ 9 endpoint REST API, luồng đăng ký, đăng nhập, xoay vòng Refresh Token (Test #8), phân trang lịch sử tin nhắn (Test #14), chặn 403 người ngoài phòng (Test #16), kiểm thử Race Condition 2 luồng đồng thời tạo phòng 1-1 (Test #17) và lưu trữ tin nhắn real-time (Test #18).
- JwtStompChannelInterceptorTest (7 bài test unit) : Kiểm thử độc lập interceptor xác thực JWT khi CONNECT, chặn token hết hạn, kiểm soát quyền khi SUBSCRIBE và chặn các destination vi phạm.
- SpringMechanismIntegrationTest (2 bài test) : Kiểm chứng bean DataSource singleton, kiểm tra ConditionEvaluationReport HikariCP và xác nhận cơ chế rollback giao dịch Spring qua proxy khi câu lệnh INSERT token thay thế bị lỗi.
- RoomServiceUnitTest (1 bài test) : Kiểm thử đơn vị bằng Mockito cho RoomService.sendMessage, xác nhận người ngoài phòng bị chặn 403 Forbidden trước khi thực hiện bất kỳ lệnh ghi nào vào CSDL.

5.2. Kết quả Kiểm thử E2E Smoke qua Mạng Thật (tools/e2e_check.py)

Kịch bản tools/e2e_check.py khởi chạy ứng dụng trên cổng tạm kết nối PostgreSQL 16 thật và điều khiển 2 client WebSocket native trao đổi tin nhắn. Kết quả đạt 15/15 checks PASS :

1. Đăng ký/đăng nhập 2 tài khoản thử nghiệm A và B.
2. Mở 2 kết nối WebSocket /ws kèm JWT header.
3. Client A gửi tin nhắn tới /app/room.7, Client B gửi tin phản hồi.
4. Cả 2 client cùng nhận đúng MessageDTO chứa ID do server cấp.
5. Đối chiếu CSDL PostgreSQL xác nhận lưu trữ đủ 2 bản ghi tin nhắn và cập nhật last_message_id.
6. Người dùng ngoài C cố tình SUBSCRIBE hoặc SEND vào phòng bị chặn lỗi 403 / ERROR frame, CSDL giữ nguyên không bị sai lệch.

5.3. Thí nghiệm SQL Lab Tách biệt (Đo đạc Tầng CSDL)

Để phân tích hiệu năng CSDL mà không làm kéo dài thời gian build test Spring Boot, nhóm xây dựng script độc lập tools/sql_lab.py thực thi trực tiếp trên PostgreSQL 16.15 (giới hạn 1 vCPU, 512 MiB RAM):

- Thí nghiệm Khảo sát Phân trang Lịch sử (get_room_messages) :
  - *Mức 1.000 tin*: Offset 0 đạt 0.715 ms; Offset 900 đạt 1.393 ms.
  - *Mức 10.000 tin*: Offset 0 đạt 1.974 ms; Offset 9.000 đạt 10.864 ms.
  - *Mức 100.000 tin*: Offset 0 đạt 18.029 ms; Offset 90.000 đạt 131.194 ms.
- *Kết luận*: Kết quả đo đạc giải thích rõ chi phí duyệt bản ghi OFFSET và câu lệnh COUNT(*) khi tập dữ liệu phình to, làm cơ sở đề xuất hướng phát triển Keyset / Cursor Pagination.
- Thí nghiệm Đối chứng Khóa Advisory (pg_advisory_xact_lock) :
  - Chạy 3 lượt thử nghiệm với 2 worker gọi hàm đồng thời (có chèn delay nhân tạo pg_sleep(0.15) để mở cửa sổ tranh chấp).
  - *Biến thể có khóa (locked)*: 3/3 lượt đều chỉ sinh ra đúng 1 phòng chat và 2 participant (2 worker nhận cùng roomId).
  - *Biến thể bỏ khóa (unlocked)*: 3/3 lượt đều bị sinh ra 2 phòng chat trùng lặp và 4 participant.
  - *Kết luận*: Minh chứng thực nghiệm khẳng định vai trò bắt buộc của khóa tư vấn trong việc bảo đảm tính nguyên tử (idempotent) của thao tác tạo phòng 1-1.

6. Bài học Kinh nghiệm & Nhật ký Sửa lỗi (Debugging Case Studies)

Trong quá trình phát triển và rà soát hệ thống, nhóm đã ghi nhận 3 tình huống kỹ thuật tiêu biểu:

Tình huống 1: Lỗi Mất Principal giữa các Khung truyền STOMP

- Hiện tượng : Client gọi CONNECT thành công (nhận CONNECTED), nhưng khi gửi frame SUBSCRIBE lập tức bị server ngắt kết nối với lỗi ERROR.
- Nguyên nhân : Trong mã ban đầu, interceptor khởi tạo accessor mới bằng StompHeaderAccessor.wrap(message). Việc này làm đứt gãy callback gán bởi STOMP transport, dẫn đến thông tin Principal bị thất lạc ở các khung truyền tiếp theo.
- Giải pháp : Cập nhật interceptor trích xuất accessor sẵn có qua MessageHeaderAccessor.getAccessor(message, StompHeaderAccessor.class), thực hiện setUser(authentication) trực tiếp trên accessor gốc và trả về thông điệp ban đầu.

Tình huống 2: Giữ đúng totalElements khi Trang Lịch sử Rỗng

- Hiện tượng : Khi client truy vấn lịch sử tin nhắn ở một trang ngoài phạm vi (ví dụ page=5, size=10 trên phòng chỉ có 2 tin), phản hồi bị trả về totalElements = 0.
- Nguyên nhân : Hàm PL/pgSQL get_room_messages lặp giá trị total_count trên từng dòng kết quả. Khi danh sách dòng trả về rỗng, service không đọc được total_count từ mảng kết quả.
- Giải pháp : Xây dựng nhánh xử lý dự phòng trong RoomService.getRoomMessages: nếu danh sách dòng rỗng, service tự động gọi messageRepository.countByConversationIdAndDeletedFalse(roomId) để đảm bảo giữ nguyên giá trị tổng số bản ghi chính xác cho client.

Tình huống 3: Lỗi Bất đồng Hợp đồng Destination STOMP giữa Client và Backend

- Hiện tượng : Client web thử nghiệm gửi tin nhắn tới /app/chat.sendMessage nhưng backend không tiếp nhận.
- Nguyên nhân : Sự sai lệch giữa tài liệu thiết kế nháp ban đầu (/app/chat.sendMessage) và annotation mã nguồn thực tế trong RoomWebSocketController (/room.{roomId}).
- Giải pháp : Nhóm thực hiện đồng bộ dứt điểm tại commit 332d069 trên nhánh phu-tai: chuẩn hóa destination gửi thành /app/room/{roomId} và destination đăng ký nhận thành /topic/room/{roomId}, đồng thời cập nhật đồng bộ trong interceptor, controller và client app.js.

7. Đánh giá Tổng kết & Định hướng nâng cấp

Nhìn lại toàn bộ lộ trình 12 tuần, dự án NextChat Backend đã hoàn tất trọn vẹn các mục tiêu nghiên cứu cốt lõi:

1. Xây dựng thành công hạ tầng Backend REST API & WebSocket/STOMP an toàn, tin cậy trên nền Spring Boot 4.1.1 và Java 21.
2. Quản lý dữ liệu quan hệ chặt chẽ bằng PostgreSQL 16, Flyway migration và các hàm PL/pgSQL tối ưu hóa.
3. Giải quyết dứt điểm các bài toán phức tạp về concurrency (Race Condition tạo phòng 1-1) và security (Refresh Token Rotation băm SHA-256, STOMP Interceptor authentication).
4. Chứng minh tính đúng đắn bằng bộ test tự động 28 JUnit PASS và 15 E2E Smoke PASS.

Hướng Phát triển Nâng cấp trong Tương lai

- Chuyển đổi Cursor Pagination : Tái cấu trúc API lịch sử tin nhắn sang cơ chế Keyset Pagination (dựa trên created_at và id) để đạt thời gian phản hồi $O(1)$ cố định cho các màn hình cuộn vô hạn với hàng triệu bản ghi.
- Hạ tầng Redis Broker Relay : Thay thế Simple Broker trong bộ nhớ bằng Redis Pub/Sub hoặc RabbitMQ STOMP Relay, cho phép mở rộng hệ thống backend theo mô hình đa tiến trình/đa instance (Horizontal Scaling).
- Hoàn thiện Dockerfile & CI/CD : Đóng gói ứng dụng Spring Boot bằng Dockerfile multi-stage build và thiết lập pipeline GitHub Actions tự động thực thi ./mvnw clean verify cho mọi Pull Request.